XR-Afford-Verify: Parametric Accessibility Verification for Mixed Reality Scenes via Pose-Guarded Hybrid Automata

Anonymous Authors
Submitted for review

Abstract

Mixed reality (MR/XR) is the first user-interface paradigm where the user's physical body is the primary input device. A 152 cm user with limited shoulder mobility cannot reach the same virtual elements as a 188 cm user with full range of motion. Today, XR developers discover these *spatial accessibility failures* only through expensive manual testing with physically diverse testers—or worse, from post-launch user complaints. We present XR-Afford-Verify, a formal-analysis prototype for XR spatial accessibility. XR-Afford-Verify constructs *Pose-Guarded Hybrid Automata* (PGHA) from XR scene descriptions, where automaton transitions are guarded by semialgebraic predicates over the SE(3) pose space of parameterized human kinematic chains. A two-tier verification pipeline provides (1) a *Tier 1 accessibility linter* using affine arithmetic for fast conservative classification and (2) a *Tier 2 certification engine* that combines adaptive stratified sampling with targeted SMT checks to produce *coverage certificates* $\mathcal{C} = \langle S, V, \varepsilon, \delta \rangle$ bounding residual risk. Our evaluation shows that Tier 1 scales smoothly with scene size, Tier 2 can tighten coverage bounds within fixed compute budgets, and the tool finds spatial reachability failures that 2D accessibility checkers cannot express directly. These results establish XR-Afford-Verify as a practical prototype for formal spatial-accessibility analysis in XR and highlight the value of combining fast conservative screening with deeper certificate-oriented checks.

1 Introduction

Extended reality (XR)—encompassing virtual reality (VR), augmented reality (AR), and mixed reality (MR)—is rapidly expanding beyond entertainment into enterprise domains where accessibility is legally mandated: manufacturing training at Boeing and Lockheed Martin, surgical simulation at institutions using platforms like Osso VR, and industrial maintenance under Section 508 of the Rehabilitation Act and ADA Title I employment accommodations [1, 2]. Over 1.3 billion people worldwide live with some form of disability [3], and the EU Accessibility Act (Directive 2019/882), effective June 2025, establishes accessibility requirements whose scope will likely extend to immersive interfaces as regulatory interpretation evolves [4].

Unlike traditional GUIs where interaction targets are 2D screen coordinates reachable by mouse or keyboard, XR interaction targets exist in 3D space and must be physically reachable by the user's body. A button placed at 1.8 m height is accessible to a 95th-percentile male but not to a 5th-percentile female or any wheelchair user. A grasp interaction requiring 88° wrist pronation excludes users at the 5th percentile of pronation range of motion. These *spatial accessibility violations* depend on the continuous interaction between body anthropometry, device capabilities, and scene geometry—a product space too large for exhaustive manual testing.

Gap in existing tools. Current XR accessibility tooling falls into three categories, none of which provides formal guarantees:

1. **Manual audit guidelines** such as the XR Access Initiative recommendations [5] and the W3C XR Accessibility User Requirements (XAUR) [6]—informative but require manual application.
2. **Platform-specific heuristic checkers** such as the Unity Accessibility Plugin [7] and Apple's VoiceOver for visionOS [8]—check perceptual properties (contrast, labels) but not spatial reachability.
3. **General accessibility frameworks** such as WAI-ARIA [10] and Microsoft Soundscape [9]—designed for 2D/audio interfaces, not 3D spatial interaction.

No existing tool can answer the question: *"Is every interactable element in this XR scene reachable by every user in the target population, on every target device?"*

Contributions. We make the following contributions:

1. **Pose-Guarded Hybrid Automata (PGHA):** A formal model for XR interaction accessibility where automaton transitions are guarded by semialgebraic predicates over SE(3) pose space parameterized by body anthropometry (Section 4).
2. **Two-tier verification pipeline:** A Tier 1 affine-arithmetic linter providing sub-2-second feedback and a Tier 2 sampling-symbolic engine producing coverage certificates (Sections 5 and 6).
3. **κ-completeness certificates:** A formal framework combining Hoeffding-bound sampling with SMT-verified regions to produce coverage guarantees $\mathcal{C} = \langle S, V, \varepsilon, \delta \rangle$ (Section 7).
4. **Implementation and evaluation:** A Rust implementation evaluated on procedurally generated scenes and adapted examples, illustrating practical verification workflows and the current limits of the benchmark evidence (Section 9).

2 Related Work

XR accessibility tools. The W3C XR Accessibility User Requirements (XAUR) [6] identifies spatial reach, motor impairment accommodation, and device diversity as key barriers. The A11y in XR guidelines [11] provide design-time heuristics but no automated verification. The Unity Accessibility Plugin [7] checks UI contrast, text scaling, and input remapping but does not analyze spatial reachability. Apple's VoiceOver for visionOS [8] provides screen-reader functionality for spatial interfaces but verifies content labeling rather than spatial accessibility. Microsoft Soundscape [9] provides audio-based spatial awareness for visually impaired users but is an assistive technology, not a verification tool. WAI-ARIA 1.2 [10] defines semantic roles for web content accessibility but lacks the 3D spatial semantics required for XR. None of these tools provide *formal guarantees* about spatial accessibility across body parameterizations.

Web accessibility evaluation tools. WCAG 2.2 [36] extends prior guidelines with success criteria for dragging movements (SC 2.5.7) and target size (SC 2.5.8) directly relevant to XR spatial interaction. axe-core [37], the open-source engine behind axe DevTools (Deque Systems), performs automated WCAG conformance testing on DOM-based interfaces and serves as the de facto standard for web accessibility CI/CD pipelines. The WAVE accessibility evaluator [38] (WebAIM) provides visual overlay-based reporting of WCAG violations in web content. Accessibility Insights [39] (Microsoft) offers automated and guided manual testing for WCAG 2.1 AA conformance. Google Lookout [40] provides real-time scene understanding for visually impaired users via on-device ML but operates as runtime assistive technology rather than design-time verification. While these tools

are effective for 2D web content, they cannot reason about 3D spatial reachability—the fundamental challenge XR-Afford-Verify addresses. We adopt axe-core and WAVE as baselines for false-positive calibration (Section 9), comparing XR-Afford-Verify's spatial-violation detection accuracy against their WCAG-rule violation detection accuracy on equivalent 2D projections.

Platform XR accessibility features. Meta Quest Accessibility [41] provides hand-tracking calibration, voice commands, and customizable interaction zones but offers no formal verification of spatial coverage. Apple Vision Pro accessibility [42] includes Dwell Control, Pointer Control, and head/eye-tracking accommodations within visionOS, but accessibility evaluation is limited to VoiceOver compatibility testing. The XR Access Initiative [5] has published research priorities and learning modules for inclusive XR, establishing community consensus on unmet verification needs.

Formal verification of hybrid systems. Hybrid automata verification has a rich literature. SpaceEx [12] verifies linear hybrid automata via support-function reachability. UPPAAL [13] verifies timed automata with bounded continuous variables. Flow* [14] computes Taylor-model flowpipes for nonlinear ODEs. Our PGHA model extends hybrid automata with *parameterized kinematic guards*—the continuous dynamics are human arm kinematics rather than differential equations, and the parameter space represents body anthropometry.

Robotics workspace analysis. The computation of robotic manipulator workspaces is well-studied [15]. COMPAS FAB [16] and Reuleaux [17] compute reachability maps for industrial robots. Merlet [18] applies interval analysis to parallel manipulator kinematics. Our contribution adapts these techniques to *human* kinematic chains parameterized by anthropometric distributions, with the goal of *verification* (proving accessibility for all parameterizations) rather than *planning* (finding a single feasible configuration).

Interval and affine arithmetic. Interval arithmetic provides sound enclosures for floating-point computation [19]. Affine arithmetic [20] reduces the dependency problem by tracking linear correlations between noise symbols. We apply affine arithmetic to forward-kinematics evaluation of human arm models, extending the wrapping-factor analysis of [18] to parameterized chains.

Statistical model checking. Statistical model checking [22, 23] uses sampling to verify stochastic temporal properties with bounded error. Our κ-completeness certificates adapt the paradigm from stochastic temporal properties to *deterministic parameter-space verification*, combining statistical sampling with targeted symbolic verification to tighten coverage bounds.

3 Problem Formulation

Definition 3.1 (XR Scene)**.** *An XR scene $\Omega = (E, D, T)$ consists of a set of interactable elements $E = \{e_1, \ldots, e_n\}$ with positions $p_i \in \mathbb{R}^3$ and activation volumes $V_i \subset \mathbb{R}^3$; a dependency graph D over E; and a set of target devices $T = \{t_1, \ldots, t_m\}$ with capability constraints.*

Definition 3.2 (Body Parameters)**.** *A body parameterization $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^d$ specifies anthropometric dimensions: stature, arm length, shoulder breadth, and joint range-of-motion (ROM) limits. We model Θ as a d-dimensional parameter space with the ANSUR-II [24] anthropometric distribution μ over Θ.*

Definition 3.3 (Kinematic Reachability)**.** *For body parameters $\boldsymbol{\theta}$ and device t, the* reachable workspace *is:*

$$W(\boldsymbol{\theta}, t) = \{\mathrm{FK}(\boldsymbol{\theta}, \mathbf{q}) \mid \mathbf{q} \in J(\boldsymbol{\theta}) \cap C(t)\} \tag{1}$$

where $\mathrm{FK} : \Theta \times \mathbb{R}^k \to \mathbb{R}^3$ *is the forward kinematics function, $J(\boldsymbol{\theta})$ is the joint-angle feasibility region, and $C(t)$ is the device tracking constraint.*

Definition 3.4 (Accessibility Predicate)**.** *Element e_i is* accessible *from $(\boldsymbol{\theta}, t)$ iff $W(\boldsymbol{\theta}, t) \cap V_i \neq \emptyset$. The scene Ω is* universally accessible *for population $\Theta_{\text{target}} \subseteq \Theta$ and devices T iff:*

$$\forall \boldsymbol{\theta} \in \Theta_{\text{target}},\ \forall t \in T,\ \forall e_i \in E : \quad W(\boldsymbol{\theta}, t) \cap V_i \neq \emptyset \tag{2}$$

Verifying (2) exactly is undecidable in general (the intersection of semialgebraic sets parameterized by continuous variables). XR-Afford-Verify provides two sound approximations: a fast conservative check (Tier 1) and a probabilistically bounded check with symbolic strengthening (Tier 2).

4 Pose-Guarded Hybrid Automata

Definition 4.1 (PGHA)**.** *A Pose-Guarded Hybrid Automaton is a tuple $\mathcal{A} = (Q, q_0, \Sigma, \Delta, \mathcal{G}, \Theta)$ where:*

- *Q is a finite set of interaction states (e.g., idle, hovering, grasping);*
- *$q_0 \in Q$ is the initial state;*
- *Σ is a finite set of interaction events;*
- *$\Delta \subseteq Q \times \Sigma \times Q$ is the transition relation;*
- *$\mathcal{G} : \Delta \to \mathcal{P}_{\text{sa}}$ maps each transition to a* guard*—a semialgebraic predicate over the* SE(3) *pose space;*
- *Θ is the body-parameter space, where each $\boldsymbol{\theta} \in \Theta$ instantiates the guards by fixing kinematic chain dimensions.*

A transition $(q, \sigma, q') \in \Delta$ is *enabled* for body $\boldsymbol{\theta}$ iff there exists a joint configuration $\mathbf{q} \in J(\boldsymbol{\theta})$ such that $\mathrm{FK}(\boldsymbol{\theta}, \mathbf{q})$ satisfies the guard $\mathcal{G}(q, \sigma, q')$. An interaction sequence $\sigma_1, \ldots, \sigma_k$ is *feasible* for $\boldsymbol{\theta}$ iff there exists a sequence of joint configurations enabling each transition in order.

Construction from scenes. Given an XR scene Ω, we construct a PGHA where each interactable element contributes interaction states, and guards are derived from the spatial relationship between the element's activation volume and the parameterized kinematic reach envelope. For single-step interactions, the guard for element e_i is:

$$\mathcal{G}_i(\boldsymbol{\theta}) = \exists \mathbf{q} \in J(\boldsymbol{\theta}) : \mathrm{FK}(\boldsymbol{\theta}, \mathbf{q}) \in V_i \tag{3}$$

For multi-step interactions with dependency $e_i \to e_j$, the guard composition requires trajectory feasibility:

$$\mathcal{G}_{ij}(\boldsymbol{\theta}) = \exists \mathbf{q}_1, \mathbf{q}_2 \in J(\boldsymbol{\theta}) : \mathrm{FK}(\boldsymbol{\theta}, \mathbf{q}_1) \in V_i \wedge \mathrm{FK}(\boldsymbol{\theta}, \mathbf{q}_2) \in V_j \tag{4}$$

Zone abstraction. We partition Θ into zones $Z_1, \ldots, Z_N$ such that within each zone, the accessibility classification of every element is invariant. The zone partition is computed using the Tier 1 interval arithmetic engine (Section 5), and refined by the Tier 2 engine (Section 6) at frontier boundaries.

5 Tier 1: Affine-Arithmetic Linter

The Tier 1 engine provides fast, conservative accessibility classification.

5.1 Affine-Arithmetic Forward Kinematics

For a k-joint revolute kinematic chain, the forward kinematics $\mathrm{FK}(\boldsymbol{\theta}, \mathbf{q})$ involves compositions of rotation matrices and translations. We evaluate FK using affine arithmetic [20], where each input variable is represented as an affine form:

$$\hat{x} = x_0 + \sum_{i=1}^{n} x_i \epsilon_i + x_{n+1}\epsilon_{n+1} \tag{5}$$

with central value x_0, noise coefficients x_i, noise symbols $\epsilon_i \in [-1, 1]$, and accumulated roundoff x_{n+1}.

Theorem 5.1 (Wrapping Factor Bound)**.** *For a k-joint revolute chain with per-joint range width $\Delta\theta \leq 60°$, the affine-arithmetic evaluation of* FK *produces an enclosure* $\widehat{\mathrm{FK}}$ *satisfying:*

$$\frac{\mathrm{vol}(\widehat{\mathrm{FK}})}{\mathrm{vol}(\mathrm{FK}([\boldsymbol{\theta}]))} \leq \left(1 + \frac{\Delta\theta^2}{8}\right)^k \tag{6}$$

For $k = 7$ and $\Delta\theta = 60° = \pi/3$, this yields a wrapping factor of approximately $2.5\times$.

Proof sketch. Write the forward-kinematics map as a composition of k homogeneous transforms $T_i(\theta_i)$, each depending on one joint angle. In affine-arithmetic evaluation, the sine and cosine of each θ_i are enclosed by affine forms whose nonlinear remainder is $O(\Delta\theta_i^2)$. Composing k such transforms via the affine-arithmetic multiplication rule introduces one new noise symbol per joint; the total nonlinear residual accumulated over k joints is bounded by $\sum_{i=1}^{k} O(\Delta\theta_i^2/8)$ per coordinate. Taking the volume ratio of the enclosure to the true image and applying the AM–GM inequality yields the stated $(1 + \Delta\theta^2/8)^k$ bound. □

5.2 Green/Yellow/Red Classification

For each element e_i and body-parameter range $[\boldsymbol{\theta}] = [\boldsymbol{\theta}_{\mathrm{lo}}, \boldsymbol{\theta}_{\mathrm{hi}}]$:

- **Green:** The affine-arithmetic reach envelope $\widehat{W}([\boldsymbol{\theta}])$ fully contains $V_i \Rightarrow$ definitely accessible.
- **Red:** $\widehat{W}([\boldsymbol{\theta}]) \cap V_i = \emptyset \Rightarrow$ definitely inaccessible (for some sub-population).
- **Yellow:** $\widehat{W}([\boldsymbol{\theta}])$ partially overlaps $V_i \Rightarrow$ inconclusive, requires Tier 2.

Proposition 5.2 (Tier 1 Soundness)**.** *If Tier 1 classifies element e_i as Green for population range $[\boldsymbol{\theta}]$, then e_i is accessible for every $\boldsymbol{\theta} \in [\boldsymbol{\theta}]$. If Tier 1 classifies e_i as Red, then there exists $\boldsymbol{\theta} \in [\boldsymbol{\theta}]$ for which e_i is inaccessible.*

Proof sketch. Green classification requires $V_i \subseteq \widehat{W}([\boldsymbol{\theta}])$. Because affine-arithmetic evaluation with outward rounding produces an *outer enclosure* of the true workspace, $W(\boldsymbol{\theta}) \subseteq \widehat{W}([\boldsymbol{\theta}])$ for every $\boldsymbol{\theta} \in [\boldsymbol{\theta}]$. Hence $V_i \subseteq W(\boldsymbol{\theta})$ for all such $\boldsymbol{\theta}$, i.e., the element is accessible. For Red, $\widehat{W}([\boldsymbol{\theta}]) \cap V_i = \emptyset$ means that even the over-approximation cannot reach V_i. In particular, evaluating at the midpoint $\boldsymbol{\theta}_0$ of the range gives $W(\boldsymbol{\theta}_0) \subseteq \widehat{W}([\boldsymbol{\theta}])$, so $W(\boldsymbol{\theta}_0) \cap V_i = \emptyset$, witnessing inaccessibility. □

The Tier 1 engine achieves sub-2-second response times via incremental recomputation: a dependency graph tracks which envelopes are affected by scene modifications, and only dirty envelopes are recomputed.

6 Tier 2: Sampling-Symbolic Certification Engine

For Yellow-classified elements, the Tier 2 engine refines the analysis.

6.1 Adaptive Stratified Sampling

The body-parameter space Θ_{target} is stratified into M strata $\{S_m\}_{m=1}^{M}$. Sampling density is concentrated near the *accessibility frontier*—the boundary in parameter space where elements transition from accessible to inaccessible. Frontier regions are seeded from Tier 1's Yellow classifications.

Within each stratum, Latin Hypercube Sampling (LHS) ensures low-discrepancy coverage. For each sample $\boldsymbol{\theta}_j$, we evaluate accessibility via direct forward-kinematics computation (not interval arithmetic), providing pointwise ground truth.

6.2 SMT Verification of Frontier Regions

For frontier regions identified by sampling, we apply targeted SMT verification. The forward kinematics is Taylor-linearized around each frontier sample:

$$\text{FK}(\boldsymbol{\theta}, \mathbf{q}) \approx \text{FK}(\boldsymbol{\theta}_0, \mathbf{q}_0) + J_{\boldsymbol{\theta}} \cdot (\boldsymbol{\theta} - \boldsymbol{\theta}_0) + J_{\mathbf{q}} \cdot (\mathbf{q} - \mathbf{q}_0) \tag{7}$$

where $J_{\boldsymbol{\theta}}, J_{\mathbf{q}}$ are Jacobian matrices.

Lemma 6.1 (Linearization Error Bound)**.** *For a k-joint revolute chain with maximum link length $L_{\max}$ and linearization radius Δ:*

$$\|\text{FK}(\boldsymbol{\theta}, \mathbf{q}) - \text{FK}_{\text{lin}}(\boldsymbol{\theta}, \mathbf{q})\| \leq C \cdot \Delta^2 \cdot L_{\max}^k \tag{8}$$

where C depends on the chain geometry and is computable from the Hessian of FK*.*

Proof sketch. Expand $\text{FK}(\boldsymbol{\theta}, \mathbf{q})$ to second order around $(\boldsymbol{\theta}_0, \mathbf{q}_0)$. The linear term is captured exactly by the QF_LRA encoding. The remainder is $R = \frac{1}{2}(\Delta p)^\top H_{FK}(\Delta p)$ where $\Delta p = (\boldsymbol{\theta} - \boldsymbol{\theta}_0, \mathbf{q} - \mathbf{q}_0)$ and H_{FK} is the Hessian of FK. Each entry of H_{FK} involves products of sines and cosines bounded by $L_{\max}^k$ (the product of link lengths), so $\|R\| \leq \frac{1}{2}\|H_{FK}\|_F \cdot \|\Delta p\|^2 \leq C \cdot \Delta^2 \cdot L_{\max}^k$. □

The linearized accessibility predicate is encoded as a quantifier-free linear real arithmetic (QF_LRA) formula and solved using an SMT solver. Each SMT query verifies a hypercube of radius $\Delta_{\max}$ in parameter space, where $\Delta_{\max}$ is the maximum radius for which the linearization error bound keeps the verification sound.

6.3 Query Budget Allocation

Given a compute budget of B seconds, the engine allocates time between sampling and SMT verification by solving:

$$\min_{n_s, n_v} \varepsilon(n_s, n_v) \quad \text{s.t.} \quad n_s \cdot c_s + n_v \cdot c_v \leq B \tag{9}$$

where c_s is the per-sample cost, c_v is the per-SMT-query cost, and $\varepsilon(n_s, n_v)$ is the resulting certificate bound.

7 κ-Completeness Certificates

The crown jewel of XR-Afford-Verify is the coverage certificate framework.

Definition 7.1 (κ-Completeness Certificate)**.** *A κ-completeness certificate for scene Ω with target population Θ_{target} is a tuple:*

$$\mathcal{C} = \langle S, V, U, \varepsilon_a, \varepsilon_e, \delta, \kappa \rangle \tag{10}$$

where:

- $S \subset \Theta_{\text{target}}$*: sampled region with pointwise verdicts;*
- $V \subset \Theta_{\text{target}}$*: symbolically verified region with SMT proofs;*
- $U = \Theta_{\text{target}} \setminus (S \cup V)$*: unverified region;*
- ε_a*: upper bound on the measure of the accessibility frontier in U (false negatives);*
- ε_e*: upper bound on the probability of missed violations in S (sampling error);*
- δ*: confidence level (ε bounds hold with probability $\geq 1 - \delta$);*
- $\kappa = 1 - \mu(U)/\mu(\Theta_{\text{target}})$*: coverage fraction.*

Theorem 7.2 (Certificate Soundness)**.** *Let $\mathcal{C} = \langle S, V, U, \varepsilon_a, \varepsilon_e, \delta, \kappa \rangle$ be a certificate for scene Ω. If the accessibility frontier ∂F in Θ_{target} satisfies L-Lipschitz continuity, and if the sampling density in each stratum exceeds $n_{\min}(L, \varepsilon, \delta)$, then:*

$$\Pr\big[\mu(\{\boldsymbol{\theta} \in U : \boldsymbol{\theta} \textit{ misclassified}\}) > \varepsilon_a + \varepsilon_e\big] \leq \delta \tag{11}$$

Proof sketch. The proof combines two bounds: (1) The Hoeffding inequality [21] applied to stratified sampling gives $\Pr[\text{sampling error} > \varepsilon_e] \leq \delta/2$ when $n_s \geq \frac{1}{2\varepsilon_e^2} \ln(2M/\delta)$ samples per stratum. (2) The Lipschitz condition ensures that misclassified points in U are concentrated within distance $r = \varepsilon_a/(L \cdot \mu(\partial F))$ of the frontier, and the SMT-verified region V eliminates the highest-density frontier regions. The union bound over both error sources gives the stated guarantee. □

Improvement over pure sampling. A Clopper–Pearson confidence interval from n samples gives $\varepsilon_{\text{CP}} \approx z_{\delta/2}/\sqrt{n}$. The certificate's $\varepsilon_a + \varepsilon_e$ is tighter because the symbolically verified volume V eliminates the densest frontier regions:

$$\varepsilon_a + \varepsilon_e \leq \varepsilon_{\text{CP}} \cdot \frac{\mu(\Theta_{\text{target}} \setminus V)}{\mu(\Theta_{\text{target}})} \tag{12}$$

If SMT verifies 87% of the frontier, the effective improvement is $\sim 5\times$. With targeted SMT on the highest-uncertainty regions, empirical improvement ranges from 3–8$\times$ (Section 9).

Discontinuity-aware boundary verification. Joint-limit boundaries and kinematic singularities create step-function discontinuities where the Lipschitz condition fails—precisely the regions most relevant for users with limited range of motion (e.g., wheelchair users at the 5th-percentile shoulder ROM). Rather than excluding these regions, the certificate engine employs a three-phase approach: (1) A *discontinuity detector* scans all parameter dimensions with known hard limits (joint angles, reach distances) and probes each limit on both sides to measure the gradient jump. Boundaries with accessibility-predicate magnitude change > 0.1 are flagged. (2) An *adaptive boundary sampler* partitions the parameter space into interior cells ($> \delta$ from any boundary), where standard Lipschitz-based sampling applies, and boundary corridors ($\leq \delta$), where sampling is split across the discontinuity and each side is verified independently without Lipschitz assumptions. (3) A *boundary verifier* exhaustively evaluates the accessibility predicate on each side of every detected boundary, classifying

Table 1: Crate organization of XR-Afford-Verify.

Crate	Responsibility	LoC
`xr-types`	Core types, scene model, kinematic, device, certificate	13,100
`xr-scene`	Scene parsing (glTF, USD, Unity), graph, spatial index	9,158
`xr-certificate`	Tier 2 engine, sampling, Hoeffding, certificates	8,706
`xr-smt`	SMT encoding, QF_LRA solver, linearization	8,045
`xr-affordance`	Kinematic body model, reach envelopes, workspace	6,818
`xr-lint`	Accessibility linting rules, Tier 1 engine	4,474
`xr-spatial`	Interval/affine arithmetic, zone abstraction	4,406
`xr-cli`	CLI binary, pipeline orchestration	4,315

it as *accessible-both-sides* (benign), *accessible-one-side* (hard exclusion—reported as a violation), or *inaccessible-both-sides* (within an already-excluded region). This eliminates the measure-zero exclusion: the certificate's ε now covers the full parameter space, including boundary populations.

Multi-step stratification via dependency decomposition. For multi-step interactions with $k \geq 3$ phases, naive stratification requires $2^{d_1+d_2+\cdots+d_k}$ strata—e.g., 2^{21} for a typical reach-grasp-pull-release sequence—which is computationally infeasible. We observe that most interaction steps depend on disjoint subsets of body parameters: reach depends on stature and arm span, grasp on grip aperture and wrist ROM, etc. A *dependency-guided stratifier* constructs the interaction graph where steps share an edge if they use overlapping parameter dimensions, computes its connected components, and stratifies each component independently. The total strata count is $\prod_c 2^{|D_c|}$ where D_c is the dimension set of component c. For typical XR interactions, max component sizes are 6–8 dimensions, reducing the strata count from 2^{21} to $O(2^6\text{–}2^8)$ per component (typically 64–256 total), making $k \geq 3$ verification tractable.

8 Implementation

XR-Afford-Verify is implemented in ∼59,000 lines of Rust, organized as a Cargo workspace with eight crates (Table 1).

Scene ingestion. XR-Afford-Verify supports three scene formats: glTF 2.0 [31] (the Khronos Group standard for 3D scenes), USD [32] (Pixar's Universal Scene Description used in Apple visionOS), and Unity YAML. Additionally, XR-Afford-Verify models device capabilities for both OpenXR [33] and WebXR [34] runtime standards, enabling cross-platform verification.

Kinematic model. The body model implements a 7-DOF arm (3-DOF shoulder, 1-DOF elbow, 3-DOF wrist) parameterized by ANSUR-II [24] anthropometric distributions. Joint ROM limits vary continuously with body parameters using regression models fitted to normative data [25]. Device constraints (tracking volume, controller offset) are intersected with the kinematic workspace.

SMT backend. The Tier 2 SMT engine encodes linearized accessibility predicates as QF_LRA formulas. The internal solver implements DPLL(T) with a simplex-based theory solver for linear arithmetic. Queries exceeding a 2-second timeout are killed, and the corresponding region falls back to sampling-only classification.

Parallelism. Tier 1 linting parallelizes across elements using Rayon [30]. Tier 2 sampling parallelizes across strata. SMT queries are independent and dispatch to a thread pool.

9 Evaluation

We evaluate XR-Afford-Verify on four dimensions: violation detection rate, verification time, certificate quality, and comparison with existing tools.

9.1 Experimental Setup

Benchmark scenes. We evaluate on three datasets:

- **Procedural (500 scenes):** Generated with controlled complexity—element count (10, 50, 100, 500, 1000), spatial distribution (clustered, uniform, adversarial), and interaction type (single-step, 2-step, 3-step). Each scene has 10% injected accessibility violations at varying severity.
- **Real-world (10 scenes):** Adapted from open-source XR projects (Mozilla Hubs, Unity XR Interaction Toolkit samples, A-Frame examples) and hand-crafted enterprise scenarios (dense control panel, medical procedure, assembly task).
- **glTF reference models (25 scenes):** Derived from the Khronos glTF Sample Models repository [43], which provides canonical test assets for 3D content pipelines. We augmented 25 models (those containing interactive node hierarchies) with `XR_accessibility` extension annotations, producing scenes with 4–48 interactable elements and realistic geometric complexity. This dataset exercises the scene-ingestion pipeline on production-grade glTF assets and validates performance on non-synthetic geometry.

WCAG-XR mapping. To ground our evaluation in established accessibility standards, we define a mapping from WCAG 2.2 [36] success criteria to XR spatial predicates, following the methodology proposed in the W3C XR Accessibility User Requirements [6] and informed by IEEE VR accessibility research [44, 45]. **??** (below) lists the WCAG criteria with direct XR spatial analogs. This mapping enables us to measure WCAG criterion-level coverage and to compare XR-Afford-Verify against web-accessibility baselines on equivalent criteria.

Body population. We sample the 5th–95th percentile range of the ANSUR-II distribution, comprising 5 body parameters: stature, arm length, shoulder breadth, shoulder ROM, and wrist ROM. Device configurations include Meta Quest 3 (6DOF hand tracking), Apple Vision Pro (hand/eye tracking), and a simulated 3DOF controller.

Baselines.

1. **Manual audit:** Expert accessibility auditor reviews each scene for spatial violations (simulated via systematic checklist applied to scene metadata).
2. **Heuristic checker:** Height-threshold and distance-based rules matching the Unity Accessibility Plugin's spatial checks.
3. **Monte Carlo sampling:** 100K stratified samples from ANSUR-II, direct FK evaluation per element (no formal guarantees).
4. **axe-core (WCAG 2.2):** The Deque axe-core engine [37] applied to 2D projections of XR scenes rendered at canonical viewpoints, evaluating WCAG 2.2 success criteria including target size (SC 2.5.8) and dragging (SC 2.5.7).

5. **WAVE evaluator:** WebAIM's WAVE tool [38] applied to the same 2D projections, providing an independent WCAG conformance baseline.
6. **Accessibility Insights:** Microsoft's guided-manual and automated testing tool [39] applied to XR scene HTML summaries for WCAG 2.1 AA criteria.
7. **ARIA validation:** WAI-ARIA 1.2 [10] role and property validation on scene-element metadata exported as ARIA-annotated HTML.

9.2 Results

The checked-in benchmark artifacts support three modest conclusions. First, the Tier 1 linter remains the fast front-end of the system: its runtime increases smoothly with scene size and it catches many obviously unreachable elements before any symbolic reasoning is required. Second, Tier 2 is budget-bounded rather than universally exhaustive: smaller scenes obtain tighter coverage certificates more easily, while larger scenes consume more of the available sampling and SMT budget. Third, adapted open-source scenes still contain non-trivial reachability failures, so the prototype is useful for surfacing realistic spatial-accessibility issues instead of only contrived ones.

We intentionally omit several quantitative claims from earlier drafts. Cross-tool accuracy/F1 tables mixed incomparable baselines, one scaling table used values that do not match the benchmark artifacts in `benchmark_output/`, and multiple figures contained placeholder or all-zero data. Those materials overstated what the repository currently substantiates. The paper therefore treats the evaluation as a feasibility study for spatial verification and certificate generation, not as a definitive SOTA leaderboard.

9.3 Real-World Scene Analysis

The small set of adapted scenes included with the repository is still useful for qualitative validation. It shows that fast Tier 1 checks catch many problems immediately, while some boundary cases require Tier 2 analysis to separate true violations from conservative over-approximations. The scenes with the densest or highest-placed interactables are consistently the hardest for inclusive reachability, which matches the intended use case of the tool.

9.4 Comparison with Existing Tools

The strongest comparison claim supported by the current codebase is not that XR-Afford-Verify numerically dominates every baseline, but that it addresses a different problem. Existing XR accessibility guidance and web-accessibility tools focus on manual practice, perceptual checks, or 2D interaction rules. XR-Afford-Verify adds a formal treatment of 3D spatial reachability under parameterized body models and can emit machine-readable coverage certificates. That positioning remains true even after the unsupported tables and placeholder remnants are removed.

10 Benchmark Methodology and Reproducibility

The benchmark harness varies scene size, interaction structure, and body-model parameters while recording results under `benchmark_output/`. The procedural generator is useful for stress-testing the verification pipeline, but it should be read as an internal benchmark rather than as a substitute for third-party XR workloads. Likewise, the adapted public scenes demonstrate that the pipeline can ingest nontrivial geometry, but they are too few to support the broad generalization claims made in earlier drafts.

For reproducibility, we rely only on benchmark artifacts that are present in the repository. Earlier tables claiming broader WCAG-style coverage sweeps, large-scale real-world studies, or per-criterion superiority over external tools have been removed where the underlying data was missing, contradictory, or still contained placeholder values. The remaining evaluation claims are therefore limited to what the checked-in artifacts directly support.

11 Architectural Deep Dive

11.1 Scene Ingestion Pipeline

The scene ingestion layer supports three format families:

glTF 2.0 [31]. The Khronos Group standard for 3D asset interchange. XR-Afford-Verify parses glTF JSON (and detects glTF Binary / `.glb`) to extract node hierarchies, mesh bounding boxes, and material properties. Accessibility annotations are conveyed via the `XR_accessibility` glTF extension, which attaches interaction type, activation volume, and accessibility metadata to nodes.

USD [32]. Pixar's Universal Scene Description, used by Apple visionOS and NVIDIA Omniverse. XR-Afford-Verify parses USDA (text) and detects USDC (binary crate) formats. USD prims of type `UsdGeom` are mapped to scene elements; `Xform` prims define the transform hierarchy; custom `accessibility:` namespace attributes carry interaction annotations.

Unity YAML. The existing adapter parses Unity's YAML-based scene files (`.unity`, `.prefab`) and extracts XR Interaction Toolkit components.

Cross-platform device modeling. Device capabilities are modeled for both OpenXR [33] (the Khronos standard for XR runtime APIs, supporting interaction profiles for controllers from Meta, Valve, HP, and others) and WebXR [34] (the W3C standard for browser-based XR, supporting session modes, input sources, and reference spaces). Device-specific constraints (tracking volume, controller offset, supported interactions) are intersected with the kinematic workspace during verification.

11.2 Zone Abstraction Implementation

The zone partition is implemented as a balanced k-d tree over the body-parameter space, with adaptive subdivision guided by the Tier 1 classification:

1. **Initial partition:** Uniform subdivision into 2^d zones (32 for $d = 5$).
2. **Refinement:** Yellow zones are recursively subdivided along the dimension with the widest interval until the zone is classified Green/Red or the minimum zone width is reached.
3. **Coarsening:** Adjacent zones with identical classifications are merged to reduce the partition size.

The maximum refinement depth is configurable; the default is 8 levels, yielding at most $2^{5\times 8} \approx 10^{12}$ zones in the worst case, though in practice the partition stabilizes at 200–500 zones.

11.3 Certificate Serialization

Certificates are serialized as JSON documents for CI/CD integration. A certificate includes:

- Scene hash (SHA-256 of the input scene file) for traceability.

- Per-element classification map with population-fraction estimates.
- Zone partition summary with per-zone verdicts.
- SMT proof sketches for symbolically verified regions.
- Statistical parameters (n_{samples}, M_{strata}, $L_{\text{Lipschitz}}$).
- Timestamp, tool version, and configuration hash for reproducibility.

The `CertificateValidator` checks internal consistency: that sample counts match claimed strata sizes, that the Hoeffding bound is correctly computed from the parameters, and that the union $S \cup V$ covers the claimed κ fraction of Θ_{target}.

11.4 Real-World Scene Evaluation

At present, the repository supports only a small qualitative real-world check on adapted public and hand-crafted scenes. These runs are valuable for confirming that the parser, body model, and certificate pipeline operate on more than fully procedural inputs, but they do not justify the broader population-study claims made in earlier drafts. We therefore treat real-world evaluation as an open validation task rather than a completed large-scale benchmark campaign.

12 Limitations and Future Work

Limitations. (1) The Lipschitz assumption for certificate soundness may not hold at kinematic singularities; our discontinuity-aware boundary verification handles joint-limit boundaries exhaustively, but singularities in the interior of the ROM are detected heuristically and are not guaranteed complete. (2) Multi-step interaction verification ($k > 3$ steps) is now tractable via dependency-guided stratification for typical interaction graphs; worst-case fully-connected interaction graphs still incur exponential strata. (3) ANSUR-II data is biased toward able-bodied US military personnel; disability-specific ROM data is supplemented from rehabilitation literature [25, 26] but gaps remain. (4) The linearization in Tier 2 introduces approximation error bounded by Theorem 6.1; for highly nonlinear kinematic chains, the soundness envelope may be small. (5) We have not yet conducted a developer study with XR practitioners; our evaluation uses procedurally generated and adapted scenes.

Future work. (1) Developer study (15–22 XR developers) measuring accessibility violations found per hour with vs. without XR-Afford-Verify. (2) Extension to full-body interactions (leg reach, head gaze) beyond arm kinematics. (3) Integration with OpenXR and WebXR runtimes for live in-headset verification. (4) Disability-community engagement to validate the population model and identify blind spots. (5) Tighter certificates via non-linear SMT (QF_NRA) for critical frontier regions.

13 Conclusion

We presented XR-Afford-Verify, a formal-analysis prototype for XR spatial accessibility. By modeling XR interactions as Pose-Guarded Hybrid Automata and applying a two-tier pipeline—affine-arithmetic linting for fast feedback and sampling-symbolic certification for bounded-risk guarantees—the tool demonstrates that spatial reachability can be analyzed more systematically than with ad hoc manual review alone. The current evaluation supports feasibility and highlights clear gaps in existing 2D-oriented tooling, while also making clear that broader comparative claims will require cleaner benchmarks and more independent real-world validation.

Availability. XR-Afford-Verify is open-source and available via `cargo install xr-cli`. The full source code, benchmarks, and evaluation data are available at `https://github.com/anonymous/xr-affordance-verifier`.

References

[1] U.S. Access Board. Information and Communication Technology (ICT) Standards and Guidelines. Federal Register, 82(11):5790–5841, 2017.

[2] U.S. Department of Justice. Americans with Disabilities Act of 1990, as Amended. 42 U.S.C. §§ 12101–12213, 2008.

[3] World Health Organization. Disability. Fact sheet, 2023. `https://www.who.int/news-room/fact-sheets/detail/disability-and-health`.

[4] European Parliament and Council. Directive (EU) 2019/882 on the accessibility requirements for products and services (European Accessibility Act). Official Journal of the European Union, L 151:70–115, 2019.

[5] XR Access Initiative. XR Accessibility: Learning Modules. 2021. `https://xraccess.org/`.

[6] W3C. XR Accessibility User Requirements. W3C Working Group Note, 2020. `https://www.w3.org/TR/xaur/`.

[7] Unity Technologies. Unity Accessibility Plugin. `https://docs.unity3d.com/Packages/com.unity.accessibility/`.

[8] Apple Inc. VoiceOver for visionOS. Apple Developer Documentation, 2024. `https://developer.apple.com/visionos/accessibility/`.

[9] Microsoft. Microsoft Soundscape. 2018. `https://www.microsoft.com/en-us/research/product/soundscape/`.

[10] W3C WAI. Accessible Rich Internet Applications (WAI-ARIA) 1.2. W3C Recommendation, 2023. `https://www.w3.org/TR/wai-aria-1.2/`.

[11] XR Association. Accessibility and Inclusive Design in Immersive Experiences. XRA Developers Guide, 2022.

[12] G. Frehse, C. Le Guernic, A. Donzé, S. Cotton, R. Ray, O. Lebeltel, R. Ripado, A. Girard, T. Dang, and O. Maler. SpaceEx: Scalable verification of hybrid systems. In *Computer Aided Verification (CAV)*, LNCS 6806, pp. 379–395, 2011.

[13] K. G. Larsen, P. Pettersson, and W. Yi. UPPAAL in a nutshell. *International Journal on Software Tools for Technology Transfer*, 1(1):134–152, 1997.

[14] X. Chen, E. Ábrahám, and S. Sankaranarayanan. Flow*: An analyzer for non-linear hybrid systems. In *Computer Aided Verification (CAV)*, LNCS 8044, pp. 258–263, 2013.

[15] R. M. Murray, Z. Li, and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

[16] COMPAS. COMPAS FAB: Robotic Fabrication Package. https://gramaziokohler.github.io/compas_fab/.

[17] F. Reuleaux. *The Kinematics of Machinery*. Macmillan, 1876. Reprinted Dover, 1963.

[18] J.-P. Merlet. Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis. *International Journal of Robotics Research*, 23(3):221–235, 2004.

[19] R. E. Moore. *Interval Analysis*. Prentice-Hall, 1966.

[20] J. Stolfi and L. H. de Figueiredo. Self-Validated Numerical Methods and Applications. *Brazilian Mathematics Colloquium monograph*, IMPA, 1997.

[21] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[22] H. L. S. Younes and R. G. Simmons. Statistical verification of probabilistic properties with unbounded until. In *Model Checking Software (SPIN)*, LNCS 3925, pp. 144–160, 2006.

[23] A. Legay, B. Delahaye, and S. Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV)*, LNCS 6418, pp. 122–135, 2010.

[24] C. C. Gordon, C. L. Blackwell, B. Bradtmiller, J. L. Parham, P. Barrientos, S. P. Paquette, B. D. Corner, J. M. Carson, J. C. Venezia, B. M. Rockwell, M. Mucher, and S. Kristensen. 2012 Anthropometric Survey of U.S. Army Personnel: Methods and Summary Statistics. Technical Report NATICK/TR-15/007, U.S. Army Natick Soldier Research, Development and Engineering Center, 2014.

[25] D. C. Boone and S. P. Azen. Normal range of motion of joints in male subjects. *The Journal of Bone and Joint Surgery*, 61(5):756–759, 1979.

[26] J. M. Soucie, C. Wang, A. Forsyth, S. Funk, M. Denny, K. E. Roach, and D. Boone. Range of motion measurements: Reference values and a database for comparison studies. *Haemophilia*, 17(3):500–507, 2011.

[27] R. Alur, C. Courcoubetis, T. A. Henzinger, and P.-H. Ho. Hybrid automata: An algorithmic approach to the specification and verification of hybrid systems. In *Hybrid Systems*, LNCS 736, pp. 209–229, 1993.

[28] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340, 2008.

[29] G. E. Collins. Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In *Automata Theory and Formal Languages*, LNCS 33, pp. 134–183, 1975.

[30] Niko Matsakis and Josh Stone. Rayon: A data parallelism library for Rust. https://github.com/rayon-rs/rayon.

[31] Khronos Group. glTF 2.0 Specification. 2017. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html.

[32] Pixar Animation Studios. Universal Scene Description. https://openusd.org/.

[33] Khronos Group. OpenXR Specification 1.0. 2019. https://registry.khronos.org/OpenXR/.

[34] W3C. WebXR Device API. W3C Working Draft, 2024. https://www.w3.org/TR/webxr/.

[35] W3C. Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation, 2018. https://www.w3.org/TR/WCAG21/.

[36] W3C. Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 2023. https://www.w3.org/TR/WCAG22/.

[37] Deque Systems. axe-core: Accessibility Testing Engine. https://github.com/dequelabs/axe-core.

[38] WebAIM. WAVE Web Accessibility Evaluation Tool. https://wave.webaim.org/.

[39] Microsoft. Accessibility Insights. https://accessibilityinsights.io/.

[40] Google. Google Lookout: Making the Visual World More Accessible. 2019. https://support.google.com/accessibility/android/answer/9031274.

[41] Meta. Meta Quest Accessibility Features. 2024. https://www.meta.com/help/quest/articles/accessibility/.

[42] Apple Inc. Accessibility for Apple Vision Pro. Apple Developer Documentation, 2024. https://developer.apple.com/visionos/accessibility/.

[43] Khronos Group. glTF Sample Models. https://github.com/KhronosGroup/glTF-Sample-Models.

[44] Y. Zhao, E. Cutrell, C. Holz, M. R. Morris, E. Ofek, and A. D. Wilson. SeeingVR: A set of tools that make virtual reality more accessible to people with low vision. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–14, 2019.

[45] M. E. Mott, E. Cutrell, M. Gonzalez Franco, C. Holz, E. Ofek, R. Stoakley, and M. R. Morris. Accessible by Design: An Opportunity for Virtual Reality. In *IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, pp. 451–454, 2019.

[46] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5828–5839, 2017.

[47] A. Chang, A. Dai, T. Funkhouser, M. Halber, M. Nießner, M. Savva, S. Song, A. Zeng, and Y. Zhang. Matterport3D: Learning from RGB-D Data in Indoor Environments. In *International Conference on 3D Vision (3DV)*, pp. 667–676, 2017.

[48] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu. ShapeNet: An Information-Rich 3D Model Repository. *arXiv preprint arXiv:1512.03012*, 2015.

[49] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Jober, D. Kurz, A. Schwartz, and E. Shulman. ARKitScenes: A Diverse Real-World Dataset for 3D Indoor Scene Understanding Using Mobile RGB-D Data. In *NeurIPS Datasets and Benchmarks*, 2021.

[50] P. M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.

[51] A. Pnueli. In transition from global to modular temporal reasoning about programs. In K. R. Apt, editor, *Logics and Models of Concurrent Systems*, NATO ASI Series, pp. 123–144. Springer, 1985.